

NET-NEUTRAL AI

Product Requirements Document

Version 2.1 — Database Schema Finalised + Frontend + Credit Economy
June 2026 | Supersedes PRD v2.0

Property	Detail
Version	2.1 — database schema finalised, frontend designs confirmed, credit economy proposed
Supersedes	PRD v2.0 (June 2026) and PRD v1.0 (Hackathon Prototype)
Status	Active
Auth	Supabase — JWT, OAuth, Row Level Security
Database	Hybrid: Local SQLite per-session + Supabase Postgres lifetime
Frontend	Web app — HTML/CSS/JS (primary) or React + Tailwind CSS
Credit model	Clients earn only. Coordinators NEVER spend. Credits are non-deductible.
Credit economy	Grant/earn/revoke proposals included — pending team review before any development

1. What Is Net-Neutral AI

The one paragraph that matters

Frontier AI training costs upwards of \$100M and is accessible to fewer than five organisations globally. Net-Neutral AI is a platform that lets anyone — a student, a researcher, a small company — upload a dataset and request collaborative model training across a network of volunteer GPU contributors, without any contributor ever seeing the raw data, and without any single company controlling the infrastructure. Contributors earn credits for every round they participate in. Coordinators — people who need compute but may have no GPU of their own — use the platform entirely for free. Credits are never deducted from coordinators. This is the net neutrality principle applied to AI compute: equal access, distributed power, no gatekeepers.

1.1 Core Guarantees

- Privacy by architecture — raw data never leaves a client device. This is structural, not a policy.
- Free for coordinators — people who need compute but cannot contribute hardware are not penalised. Credits are never deducted.
- Transparent contribution — every client's compute contribution is permanently logged and publicly visible.
- Resilience — if Supabase is unreachable during a session, training continues. Credits sync later. The system never depends on cloud availability to function.

1.2 What Changed from v2.0

Area	v2.0	v2.1
Database design	Generalised — schema pending	Finalised hybrid: SQLite (session) + Supabase (lifetime)
clients table	Not specified	New table — tracks registered nodes per session with IP and status
Credit model	Ambiguous	Explicit: coordinators never spend. Clients only earn.
Credit economy	Not addressed	Proposed grant/earn/revoke model — pending team review
Frontend	Spec only	Designs confirmed — two implementation options fully spec'ed
Stats bar	Fabricated numbers	Must use real API data — fabricated numbers removed

2. Users

2.1 The Two-Sided Platform

User Type	Who They Are	Core Need	What the Platform Gives Them
Coordinator	Researchers, students, startups — need compute, may have no GPU	Train a model without hardware or \$100M budget	Free access — upload dataset, request training, receive trained model
Client	Anyone with idle GPU — gamers, students, labs	Monetise idle hardware with zero friction	Earn credits automatically — install agent, contribute, track earnings
Guest	Visitors, potential users	Understand product before committing	Public landing page, leaderboard — no sign-up required

2.2 Coordinator Journey

1. Signs up — selects Coordinator role — Supabase handles auth
2. Coordinator dashboard — active jobs, job history
3. Submits new training job — uploads dataset CSV, optionally uploads pre-trained checkpoint, sets epochs and rounds
4. System validates inputs — dataset format, checkpoint compatibility if provided
5. Training runs — live dashboard shows round progress, accuracy per round, connected nodes, leaderboard
6. Training completes — downloads trained model and accuracy report from Supabase Storage
7. No credits deducted — platform is entirely free for coordinators

2.3 Client Journey

8. Signs up — selects Client role — Supabase handles auth
9. Downloads and installs background agent — one-time OS-specific setup
10. Agent authenticates using Supabase JWT — stored locally, refreshed automatically
11. Agent runs silently — accepts training assignments when machine is idle
12. Agent trains locally — raw data never leaves the machine
13. Agent submits weight updates directly to coordinator — temp files deleted immediately after
14. Credits logged to local SQLite per round — synced to Supabase global_credits at session end
15. Client dashboard shows lifetime credits, sessions, compute contributed

3. Product Scope

3.1 Web Platform Pages

Page	Route	Auth	Purpose
Landing	/	Public	Product pitch, real stats from API, sign-up CTAs for both roles
Sign up	/signup	Public	Role selection (Coordinator or Client) then Supabase auth
Sign in	/signin	Public	Supabase auth, redirect by role after login
Coordinator dashboard	/dashboard	Coordinator	Active jobs, job history, submit new job
New job	/jobs/new	Coordinator	Dataset upload, optional checkpoint, epoch and round config
Live training	/jobs/{id}	Coordinator	Round progress, accuracy chart, nodes, leaderboard — live polling
Job results	/jobs/{id}/results	Coordinator	Download model, accuracy report, per-round metrics
Client dashboard	/client	Client	Agent status, lifetime credits, sessions participated, install guide
Agent setup	/client/setup	Client	OS-specific install instructions, auth token for agent config
Public leaderboard	/leaderboard	Public	Top client nodes by lifetime compute contributed

3.2 Background Agent

- Installable Python process — runs silently, no terminal window after setup
- Authenticates via Supabase JWT — stored securely on client machine
- Detects machine idle state — CPU and GPU usage below configured thresholds before accepting jobs
- Downloads global model and data shard from Supabase Storage at round start
- Trains locally — data shard deleted immediately after training
- Submits weight updates directly to Flask coordinator — never stored in Supabase
- Temp weight files deleted immediately after successful submission

3.3 Optional Checkpoint

Scenario	What Happens	Recommended Rounds
Coordinator uploads checkpoint	Validated against model architecture, used as round 1 starting point	5 rounds minimum

No checkpoint uploaded	System initialises TransformerClassifier with random weights	10 rounds minimum
Invalid checkpoint uploaded	State_dict keys validated — rejected with clear error if incompatible	N/A — re-upload required

4. Database Design — Finalised Hybrid Architecture

Two databases, two jobs

Local SQLite tracks everything within one training session — fast, disposable, resets between runs. Supabase Postgres accumulates lifetime credit history permanently. A sync at session end connects the two. Training never depends on Supabase being reachable — if it is down, the session completes and sync retries on next startup.

4.1 Local SQLite — database.db

Lives on coordinator machine. Resets before each training session. Three tables with enforced foreign keys.

clients — NEW in v2.1

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incrementing row ID
client_id	TEXT	NOT NULL UNIQUE	Node identifier — unique per session
ip_address	TEXT	DEFAULT 'unknown'	Recorded on /register_node
registered_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When node registered for this session
last_seen	DATETIME	DEFAULT CURRENT_TIMESTAMP	Updated on each /submit — detects stale nodes
is_active	BOOLEAN	DEFAULT 1	Set to 0 if node times out

rounds

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incrementing row ID
round_number	INTEGER	NOT NULL UNIQUE	Round index — 1 through N
started_at	DATETIME	NOT NULL	When this round began
completed_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	When FedAvg finished
clients_submitted	INTEGER	DEFAULT 0	How many nodes submitted this round
global_accuracy	REAL	DEFAULT 0.0	Model accuracy after this round (0.0–1.0)
accuracy_delta	REAL	DEFAULT 0.0	Change vs previous round

credits

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incrementing row ID
client_id	TEXT	NOT NULL, FK → clients (CASCADE)	Which node earned these credits
round	INTEGER	NOT NULL, FK → rounds (CASCADE)	Which round credits were earned in
samples_trained	INTEGER	DEFAULT 0	Samples processed this round
time_seconds	REAL	DEFAULT 0.0	Wall-clock training time
points_earned	INTEGER	DEFAULT 0	floor(samples_trained / 5)
timestamp	DATETIME	DEFAULT CURRENT_TIMESTAMP	Server time of submission

Constraints and indexes

Type	Table	Column(s)	Purpose
UNIQUE	clients	client_id	No duplicate registrations per session
UNIQUE	rounds	round_number	One record per round
UNIQUE	credits	(client_id, round)	One submission per node per round
FK CASCADE	credits	client_id → clients	Delete credits when client deleted
FK CASCADE	credits	round → rounds	Delete credits when round deleted
PRAGMA	All	foreign_keys = ON	Enforced on every connection
INDEX	credits	client_id	Fast leaderboard queries
INDEX	credits	round	Fast per-round lookups
INDEX	clients	is_active	Fast active-node filtering

4.2 Supabase Postgres — global_credits (Lifetime)

Lives in Supabase cloud. Never resets. One row per client node, accumulated across all sessions forever.

Column	Type	Constraints	Description
id	UUID	PRIMARY KEY, gen_random_uuid()	Supabase-generated unique ID
client_id	TEXT	NOT NULL UNIQUE	Matches client_id from local SQLite

total_points	BIGINT	DEFAULT 0	Lifetime accumulated credits across all sessions
total_samples	BIGINT	DEFAULT 0	Lifetime samples trained
total_rounds	INTEGER	DEFAULT 0	Total rounds participated across all sessions
last_session_at	TIMESTAMPTZ	DEFAULT NOW()	When this node last completed a session
created_at	TIMESTAMPTZ	DEFAULT NOW()	When this node first appeared in the system

RLS policy note

Current policy allows anonymous read/write — appropriate for prototype where client_id controls access. Tighten to JWT-authenticated writes before public launch.

4.3 Sync — Session End to Supabase

After round_status transitions to 'done', coordinator runs a one-time UPSERT per client into global_credits — incrementing totals if the client has participated before, inserting if first time.

Scenario	Action	Result
Client's first session	INSERT new row	client_A: 5000 pts, 25000 samples, 5 rounds
Client has existing record	UPDATE — INCREMENT all columns	client_A: 5000 + 4800 = 9800 pts lifetime
Supabase unreachable	Skip — log warning — SQLite preserved	Retry on next coordinator startup
Supabase connection fails	Non-fatal — training session unaffected	Credits safe locally

5. Frontend — Confirmed Design Direction

Design system confirmed by team

Dark background #080c10, teal accent #00e5a0 for coordinator elements, blue #0077ff for client elements. Typography: Syne 800 for display, Space Mono for labels and badges, Inter for body. Three confirmed UI flows: coordinator setup modal overlay, scanning animation during node discovery, staggered client connection reveal with animated graph edges.

5.1 Confirmed UI Flows

Flow	Screens	Key Interaction
Coordinator setup	Dashboard skeleton (blurred) → modal overlay	User enters coordinator name, IP, client count, uploads files, clicks Start Finding
Node discovery	Scanning screen — rotating pulse rings, sweep arm, cycling status text	Animated coordinator node broadcasts on local subnet
Clients connected	Network graph — coordinator centre, clients pop in staggered	Dashed animated edges, confirm button opens full dashboard
Landing page	Hero → stats bar → role cards → features grid → footer	Two role cards: Coordinator (teal CTA) and Client (blue CTA)
Live training	Round counter, accuracy chart, leaderboard, node status	Polls /status every 5s, all components update without page reload

5.2 Stats Bar — Real Data Only

Remove all fabricated numbers before any public exposure

The current design shows placeholder stats (12,400+ GPUs, 340TB, 87 jobs, 62 countries). These must be replaced with real numbers from a public /stats API endpoint before the platform goes live. Until that endpoint exists, show dashes or hide the stats bar entirely.

Stat	API Source	Display Rule
Nodes connected	COUNT(DISTINCT client_id) FROM global_credits WHERE last_session_at > NOW() - 7 days	Always visible — show 0 if no data
Total compute	SUM(total_samples) FROM global_credits	Always visible — show 0 if no data
Active training jobs	COUNT(*) FROM jobs WHERE status = 'active'	Always visible — show 0 if no data

Countries	NOT currently tracked	Remove this stat until geo data is collected
-----------	-----------------------	--

6. Authentication

6.1 Supabase Auth

- Email and password sign-up and sign-in
- OAuth — Google and GitHub recommended for developer audience
- JWT tokens — sent as Authorization header to all Flask coordinator endpoints
- Row Level Security — database rows scoped to the authenticated user
- Automatic token refresh, logout, password reset via Supabase

6.2 Role Assignment

On sign-up, user selects Coordinator or Client. Stored in Supabase user profile. Determines dashboard shown after sign-in. Role flexibility (one user holding both roles) is a future consideration not in current scope.

6.3 Agent Authentication

Background agent stores Supabase JWT locally after first login via web platform. Uses this token to authenticate with coordinator. Supabase Python client handles token refresh automatically.

7. File Lifecycle

File	Created By	Stored In	Deleted When
Dataset CSV	Coordinator	Supabase Storage	Job deleted or retention period expires
Input checkpoint .pt	Coordinator	Supabase Storage	Same as dataset
Data shard CSV	Coordinator	Supabase Storage	Immediately after client training round completes
Round checkpoint .pt	Coordinator	Coordinator local disk	Overwritten by next round checkpoint
Final output model .pt	Coordinator	Supabase Storage	When coordinator explicitly deletes
Client temp weights .pt	Client agent	Client local disk only	Immediately after POST /submit succeeds — never in Supabase
Client data shard copy	Client agent	Client local disk only	Immediately after training round completes

Zero residual data guarantee

No user data — dataset shards or weight files — should remain on any client machine after a round completes. The agent must force-delete in a finally block so deletion happens even if training crashes. This is the product's core privacy claim.

8. Credit System

8.1 Core Principle — Coordinators Never Spend

Fundamental product decision — do not reverse without full team consensus

Coordinators use the platform entirely for free. Credits are never deducted from coordinators. The platform's equity argument depends on this: someone with no hardware should be able to train a model on the same terms as someone with a data centre. Credits exist purely to reward client contributors.

8.2 Current Credit Earning — Implemented

Action	Who	Credits Earned	Where Logged
Complete a training round	Client	$\text{floor}(\text{samples_trained} / 5)$	Local SQLite credits table
Session complete	Client	Sum of all round credits	Synced to Supabase global_credits via UPSERT

8.3 Future Credit Economy — Pending Team Review

NOT in current development scope — requires team sign-off before any implementation

The proposals below are design suggestions for the team to review and approve. Nothing in this section should be built without explicit team consensus. These are documented so the thinking is not lost between discussions.

Proposed earn mechanisms

- Sign-up grant — all new users receive onboarding credits. Gives coordinators visibility into the credit system even though they currently cannot spend them.
- Referral grant — invite a new client that completes 5 sessions, earn bonus credits
- Streak bonus — complete 10 consecutive sessions without a dropout round, earn a multiplier on that session's credits
- Uptime reward — machine available for X hours per week earns a base weekly allowance
- Academic grant — verified students and researchers receive periodic free credits via institutional partnerships
- Promotional grant — time-limited grants for early adopters, hackathon participants, or event attendees

Proposed grant mechanisms (admin-controlled)

- System grant — automatic, triggered by predefined events. Sign-up bonus, referral reward, streak bonus. No admin action required.

- Admin grant — manual grant issued by administrators to individuals or groups
- Bulk grant — CSV upload of client_ids with grant amounts for batch processing

Proposed revocation mechanisms

- Adversarial weight detection — if a client's submissions are statistical outliers across multiple rounds, credits for those rounds are revoked and the node flagged for review
- Dataset abuse — coordinator uploads malformed dataset that causes client crashes. Job credits refunded, account flagged.
- Account fraud — administrators can revoke all credits and suspend an account
- Dispute resolution — a client can dispute a revocation within 30 days, reviewed by admin

Proposed future uses of credits — for coordinators

Since coordinators currently never spend credits, the long-term question is: what do credits mean for a coordinator who also runs a client agent? Some options for future consideration:

- Priority queue access — coordinators with more credits get jobs scheduled first when demand exceeds capacity
- Extended storage retention — trained models kept in Supabase Storage beyond the standard window
- Larger dataset upload limits — standard file size limit increased for high-credit coordinators
- Early access — new features unlocked for high-credit users first

9. Non-Goals

- Credits deducted from coordinators — will not be implemented without explicit team decision and PRD revision
- Arbitrary model architectures — text classification only in current scope
- Mobile client agent — desktop only
- Blockchain credit ledger — Supabase Postgres is sufficient
- Homomorphic encryption or differential privacy — v3.0 milestone
- Multi-GPU per client node — single GPU per machine
- Real-time P2P gossip — centralised coordinator only
- Role switching — one account holding both roles simultaneously

10. Success Metrics

Metric	Target	Kill Switch
Coordinator submits job end-to-end	Zero terminal steps required	Redesign submission if any CLI needed
Client agent installs and runs	Under 5 minutes setup	Simplify to manual script for v2.0 if agent fails
Training round completes	All nodes submit within 90s timeout	Reduce epochs, increase timeout
Accuracy improves across rounds	Upward trend visible on dashboard	Check checkpoint compatibility and data split
Credits sync at session end	100% of sessions sync eventually	Log failures, retry on next startup
Temp files deleted after every round	Zero residual files on client machine	Add explicit verification step in agent
Stats bar shows real data	All numbers from live API	Hide stats bar if endpoint returns zeros

11. Roadmap

Version	Status	Key Deliverables
v1.0	Complete	Hackathon prototype — local WiFi, CLI, 3 hardcoded nodes, SQLite, IMDbb classification
v2.0	Complete (docs)	Web platform scope, Supabase auth, optional checkpoint, background agent spec
v2.1	Current	Database schema finalised, frontend designs confirmed, credit economy proposed

v2.2	Next	Frontend connected to Flask coordinator. Agent v1 (manually started). Supabase auth live.
v3.0	Planned	Full background agent, differential privacy, secure aggregation, credit economy implementation
v3.1	Planned	10+ real volunteer nodes, async training, node reputation system
v4.0	Future	Arbitrary model architectures, model marketplace
v5.0	Future	Public API, community governance, patent filing

12. Open Questions

Question	Impact	Owner	Resolution
Can a user hold both Coordinator and Client roles?	High	Team	Decide before building auth flow
Which credit economy proposals does team approve?	High	Full team	Team review meeting — then PRD v2.2
Where does coordinator server deploy in production?	High	Backend	Cloud provider decision needed before agent build
How does coordinator distribute dataset shards?	High	CS + Backend	Supabase Storage streaming vs pre-split upload
Standard dataset upload size limit?	Medium	Backend	Supabase default 50MB — may need chunked upload
RLS policy tightening timeline?	Medium	Backend	Currently open — tighten before public launch